Design and Implementation of an Automated DevSecOps Pipeline using Policy-as-Code and Continuous Security Scanning

A Project Report

Submitted in partial fulfilment of the requirements for the

Award of the degree of

“Master of Computer Applications”

By

Vikas Rawat

(Registration Number: 324200509)



Centre for Distance and Online Education

LOVELY PROFESSIONAL UNIVERSITY

PHAGWARA, PUNJAB

2026

Annexure - II

Declaration by the Student

To whom-so-ever it may concern

I, Vikas Rawat, Registration Number 324200509, hereby declare that the work done by me on "Design and Implementation of an Automated DevSecOps Pipeline using Policy-as-Code and Continuous Security Scanning", is a record of original work for the partial fulfilment of the requirements for the award of the degree, Master of Computer Applications.

Vikas Rawat (Registration Number: 324200509)

Signature of the student: _____

Dated: _____ RAWAT Vikas Digitally signed by RAWAT Vikas Date: 2026.10.19 15:45:43 +05'30'

ACKNOWLEDGEMENT

I express my sincere gratitude to the Centre for Distance and Online Education, Lovely Professional University, Phagwara, for providing the academic framework, learning resources and opportunity to carry out this capstone project as part of the Master of Computer Applications programme.

I thank the faculty members and academic mentors of the programme for the guidance and evaluation that helped me to shape the topic, plan the work and present it in the prescribed form.

I also acknowledge the open-source communities and maintainers of Terraform, GitHub Actions, Checkov, TFSec and Trivy. Their tools and documentation made it possible to build and test the pipeline described in this report. Finally, I thank my family and friends for their patience and encouragement during the project.

ABSTRACT

Cloud infrastructure is now commonly created from code, which makes it fast to deploy and equally fast to replicate mistakes. A storage bucket that allows public reads or a firewall rule that exposes SSH to the whole internet can be created in seconds, and when security review happens only after deployment such weaknesses are found late. This project designs, implements and evaluates an automated DevSecOps pipeline on GitHub Actions in which security checks run on every push and control whether an AWS deployment is allowed.

The workflow has two jobs. The audit job validates Terraform code, analyses it with Checkov (policy-as-code) and TFSec, builds a container image and scans it with Trivy, failing on CRITICAL or HIGH findings. The deployment job needs the audit job to succeed; it then configures AWS credentials from repository secrets, applies the Terraform configuration in the Mumbai region (ap-south-1) and prints the public address of a showcase web server.

The pipeline was evaluated using recorded runs. An intentionally insecure configuration, containing a public-read S3 bucket and SSH open to 0.0.0.0/0, was scanned by Checkov, which reported 11 failed and 8 passed checks and ended the job with exit code 1; the remaining steps were skipped and the deployment job did not run. A corrected configuration passed all audit steps in 43 seconds and was deployed by a 45-second deployment job, giving a total run time of 1 minute 33 seconds.

The report states the configuration of each gate, including the thirteen Checkov checks suppressed in the final workflow and the options that narrow the Trivy scan, and explains that a successful run shows that the configured checks passed, not that the system is fully secure or benchmark-compliant. Limitations include the absence of a recorded failing run for the Trivy and TFSec gates.

Keywords: DevSecOps, CI/CD, GitHub Actions, Terraform, Infrastructure as Code, Policy-as-Code, Checkov, TFSec, Trivy, AWS.

TABLE OF CONTENTS

S. No.	Title	Page
1	Declaration by Student	2
2	Acknowledgement	3
3	Abstract	4
4	List of Tables	6
5	List of Figures / Charts	7
6	List of Schemes / Algorithms	8
7	List of Abbreviations	9
8	Chapter-1: Introduction	10
9	Chapter-2: Review of Literature	14
10	Chapter-3: Implementation of Project	18
11	Chapter-4: Results and Discussions	28
12	Chapter-5: Conclusion and Future Scope	38
13	Publication Details	41
14	Annexures	42
15	References	43

LIST OF TABLES

Table No.	Title	Page
Table 1.1	Project objectives and expected outputs	12
Table 2.1	Review of key literature relevant to the project	16
Table 2.2	Comparison of the security tools used in the pipeline	17
Table 3.1	Phases of the implementation	18
Table 3.2	Technology stack and versions used	19
Table 3.3	Repository organisation	20
Table 3.4	Steps of the audit job and their configuration	21
Table 3.5	Checkov checks suppressed in the final workflow	22
Table 3.6	Steps of the deployment job	24
Table 3.7	Intentionally insecure test resources used for the negative test	25
Table 4.1	Functional test results	28
Table 4.2	Checkov results for the insecure configuration (8 passed, 11 failed)	29
Table 4.3	Observed job durations	33
Table 4.4	Failed checks, corrective action and status in the final workflow	34
Table 4.5	Achievement of project objectives	35

LIST OF FIGURES / CHARTS

Figure No.	Title	Page
Figure 1.1	Shift-left security: automated checks run at commit and build time	11
Figure 3.1	Architecture of the automated DevSecOps pipeline	20
Figure 4.1	Failed audit run: Checkov annotations in GitHub Actions (11 errors)	30
Figure 4.2	Failed audit job: Step 4 (Checkov) failed; Steps 5 to 7 skipped	30
Figure 4.3	Another failed audit run: Checkov annotations (10 errors)	31
Figure 4.4	Successful workflow run summary (audit job followed by deployment job)	31
Figure 4.5	Job views of the successful run: (a) audit job, (b) deployment job	32
Figure 4.6	Live web server deployed by the pipeline (showcase page)	33
Figure 4.7	Observed job durations from the recorded runs	34

LIST OF SCHEMES / ALGORITHMS

Scheme No.	Title	Page
Scheme 3.1	Gate logic of the DevSecOps workflow	24
Scheme 3.2	Workflow definition (devsecops.yml)	26

LIST OF ABBREVIATIONS

Abbreviation	Meaning
ACL	Access Control List
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Delivery
CIS	Center for Internet Security
CLI	Command-Line Interface
CVE	Common Vulnerabilities and Exposures
DevSecOps	Development, Security and Operations
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
HCL	HashiCorp Configuration Language
HTTP	Hypertext Transfer Protocol
IaC	Infrastructure as Code
IAM	Identity and Access Management
IP	Internet Protocol
KMS	Key Management Service
OIDC	OpenID Connect
PaC	Policy-as-Code
S3	Simple Storage Service
SARIF	Static Analysis Results Interchange Format
SHA	Secure Hash Algorithm (commit identifier)
SSH	Secure Shell
TLS	Transport Layer Security

Chapter-1: Introduction

1.1 Purpose of the Project

Cloud infrastructure is no longer assembled by hand. Servers, networks, storage buckets and access rules are written as code, committed to a repository and created automatically whenever that code changes. This speed is useful, but it also means that a weak configuration can be created just as quickly and just as repeatedly as a good one. The purpose of this project is to design, build and evaluate a small but complete delivery pipeline in which security checks are an automatic part of every code change, so that an insecure infrastructure definition or a vulnerable container image is stopped before any cloud resource is created.

The work is carried out as the MCA capstone project (ECAP788) of Lovely Professional University, Centre for Distance and Online Education. It is a practical, tool-based project: the pipeline is implemented on GitHub Actions, the infrastructure is described in Terraform and deployed to Amazon Web Services (AWS), and the security gates are provided by Checkov, TFSec and Trivy. The report documents the design, the configuration actually used, and the evidence collected from the pipeline runs.

1.2 Background and Context

DevOps brought development and operations together around automation, short feedback loops and frequent releases.[1,2] Security, however, was often left as a separate activity performed late in the cycle, by a different team, close to release. DevSecOps extends the DevOps idea by treating security as a shared responsibility and by building automated security activities into the same pipeline that builds, tests and deploys the software.[3,4]

A second trend is Infrastructure as Code (IaC). Instead of creating cloud resources through a console, engineers describe the desired state in declarative files that can be reviewed, versioned and re-applied.[5,6] Terraform is one of the most widely used tools for this purpose. The benefit is repeatability; the risk is that a mistake in a template, such as a storage bucket that allows public reads or a firewall rule that exposes the SSH port to the whole internet, is reproduced exactly every time the template is applied. Studies of IaC scripts have repeatedly found recurring security weaknesses, including hard-coded secrets and overly permissive settings, which shows that the problem is not hypothetical.[7]

Containers add a third source of risk. A container image bundles an operating-system layer and application libraries, and published images are known to carry outdated packages with publicly

documented vulnerabilities.[8] If the image is built and deployed without being scanned, those vulnerabilities reach the running system unnoticed.

To make security requirements measurable, many organisations adopt published baselines such as the Center for Internet Security (CIS) Benchmark for AWS.[9] Tools such as Checkov[10] and TFSec[11] encode a large number of such rules and evaluate Terraform code against them without deploying anything, while Trivy[12] inspects container images for known vulnerabilities. Terraform[13] provisions the infrastructure and GitHub Actions[14] provides the automation platform that links the steps together. This project combines these tools into one gated workflow.

1.3 Shift-Left Security and Policy-as-Code

“Shift-left” means moving checks to the earliest practical point in the delivery process. A defect found when the developer commits the change can be corrected in minutes by the person who wrote it, whereas the same defect found after deployment may need a change request, a maintenance window and coordination between several teams. Figure 1.1 contrasts the two positions of the security check.

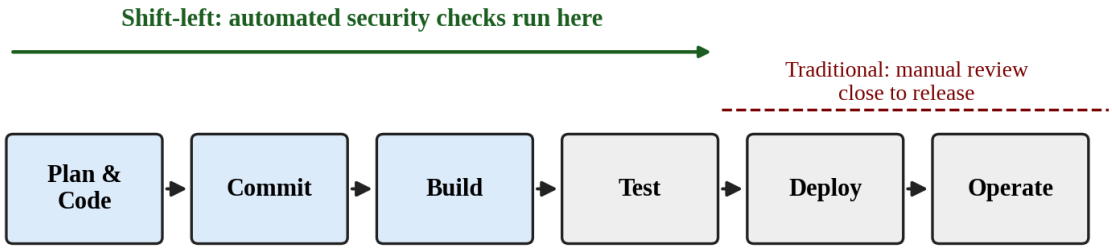


Figure 1.1: Shift-left security: automated checks run at commit and build time

Policy-as-Code (PaC) is the practice that makes shift-left realistic. A written policy such as “storage buckets must not be publicly readable” is turned into a machine-evaluated rule. The rule is run by the pipeline on every change, produces the same answer every time, and can fail the build when the policy is violated. In this project the rules are supplied by Checkov and TFSec, and the pipeline treats a violation as a failure that stops the workflow.

1.4 Problem Statement

Cloud misconfigurations such as publicly readable object storage, unencrypted data stores, administrative ports open to 0.0.0.0/0 and containers built on vulnerable base images are common and avoidable causes of exposure. They persist when security review is manual, occurs after deployment, or is run as an optional step that does not control whether deployment happens. The problem addressed by this project is therefore twofold: first, how to run infrastructure and container security checks automatically on every change; and second, how to make the deployment step depend on the outcome of those checks so that a failed check prevents any cloud resource from being created.

1.5 Aim and Objectives

The aim of the project is to design, implement and evaluate an automated DevSecOps pipeline that applies Policy-as-Code and continuous security scanning to Terraform code and a container image before an AWS deployment is allowed to proceed. The specific objectives are listed in Table 1.1.

Table 1.1: Project objectives and expected outputs

Objective	Description	Expected output
O1	Build a multi-job GitHub Actions workflow that starts automatically on every push to the main branch.	Workflow definition (devsecops.yml) and recorded runs
O2	Validate Terraform code and evaluate it with Checkov and TFSec as blocking security gates.	Gate results and findings for secure and insecure code
O3	Build a container image in the pipeline and scan it with Trivy, failing on CRITICAL and HIGH findings.	Image scan stage with explicit severity gate
O4	Make the AWS deployment job depend on the success of the audit job and deploy with Terraform in the ap-south-1 region.	Deployment job and live web server
O5	Evaluate the pipeline with a negative test (insecure configuration) and a positive test (hardened configuration) and record the evidence.	Test results, timing observations and remediation mapping

1.6 Research Questions

- Can infrastructure-as-code and container checks be combined in a single automated workflow that runs on every push?
- Does a failed policy check reliably stop the workflow before the deployment job starts?
- What does a passing pipeline run actually demonstrate, and what does it leave unproven?

1.7 Scope, Applicability and Relevance

The project addresses industrial and business issues (cloud security governance in delivery pipelines) and adds to discipline-related knowledge by documenting a reproducible, tool-based implementation. The implemented scope covers: a GitHub Actions workflow; Terraform validation; Checkov and TFSec analysis of the Terraform code; a Docker image build with a Trivy scan; and an automated Terraform deployment of a single web server in the AWS Asia Pacific (Mumbai) region, ap-south-1.

The approach is applicable to small teams, student projects and early-stage products that need a repeatable security baseline without buying a commercial platform, because every tool used is open source. It is also a teaching model: the workflow is short enough to be read in full, and each gate can be explained and tested separately. The scope does not include runtime monitoring, penetration testing, dynamic application testing, secrets scanning or an independent cloud security audit; these are listed as future work in the final chapter.

1.8 Organisation of the Report

Chapter-2 reviews the literature and tools and identifies the research gap. Chapter-3 describes the implementation: methodology, technology stack, workflow design, gate configuration and test design. Chapter-4 presents the results from the recorded pipeline runs and discusses what they do and do not show. The final chapter gives the conclusion, limitations and future scope. The publication details, annexures and references follow.

Chapter-2: Review of Literature

This chapter reviews published work and tool documentation related to DevSecOps, infrastructure-as-code security, policy-as-code and container scanning. The studies are summarised in Table 2.1 with their objective, sample or method and main findings, and the chapter closes with the research gaps that motivate the proposed work.

2.1 Evolution of DevOps and DevSecOps

Continuous delivery established the idea that every change should pass through an automated, repeatable pipeline of build, test and deployment steps, and that software should always be in a releasable state.[2] Bass, Weber and Zhu describe DevOps as a set of practices that shorten the time between a change being committed and that change running in production while maintaining quality, and they discuss how architectural decisions affect how easily such automation can be achieved.[1] In this tradition, anything that must happen before release is a candidate for automation, which naturally includes security checks.

Myrbakken and Colomo-Palacios reviewed academic and grey literature on DevSecOps and found that the term was used with differing definitions, but that most sources agreed on its core idea: integrating security into the DevOps cycle and automating it rather than leaving it to a late manual review.[3] Rajapakse and co-authors later reviewed the challenges organisations face in adopting DevSecOps and the solutions reported for them. Their review reports that adoption is hindered by practical difficulties, among them the integration of security tools into pipelines and the interpretation and handling of tool output, alongside organisational factors.[4] These findings are directly relevant to this project, which concentrates on integrating three scanners into one pipeline and on making their output control the deployment decision.

2.2 Infrastructure as Code and Its Security Implications

Morris describes infrastructure as code as the practice of defining and managing infrastructure through machine-readable definition files that are treated like application code: stored in version control, reviewed, tested and applied automatically.[5] Rahman, Mahdavi-Hezaveh and Williams carried out a systematic mapping study of IaC research and reported the topics, techniques and research areas that dominate the field. Their study shows that research on the quality and security of IaC scripts was still developing and that more work on practical tooling was needed.[6]

Rahman, Parnin and Williams examined security smells in IaC scripts, meaning code patterns that indicate a possible security weakness. By analysing scripts from open-source repositories they identified recurring categories, among them hard-coded secrets, use of HTTP without TLS and binding services to all network interfaces (0.0.0.0), and they built a static analysis tool to detect them.[7] The practical lesson for this project is that the kinds of faults used in the negative test here (public access and unrestricted network ingress) are representative of faults that really occur in IaC code and that static analysis can detect them without running the code.

2.3 Policy-as-Code, Benchmarks and Static Analysis Tools

Security benchmarks such as the CIS AWS Foundations Benchmark publish recommended configurations for cloud accounts and services.[9] A benchmark is a document; to enforce it automatically, its recommendations must be expressed as executable rules. Checkov is an open-source static analysis tool that scans IaC (including Terraform) against a large library of built-in policies, reports the identifier of each rule that passed or failed together with the file and line range of the resource, and returns a non-zero exit status when checks fail, which allows a CI system to treat the scan as a gate.[10] TFSec is a Terraform-focused scanner that parses the configuration and reports misconfigurations for the major cloud providers.[11] The two tools overlap but are not identical in rule sets, so running both widens coverage at low cost.

A property shared by these tools, and important for this report, is that they support suppressing individual checks, either inline in the code or through command-line or action parameters. Suppression is legitimate when a rule does not fit the architecture or the risk is accepted, but it also means that a “passed” result is only meaningful in relation to the list of checks that were actually evaluated. This project therefore records the suppressed checks explicitly (Chapter-3, Table 3.5).

2.4 Container Image Vulnerability Scanning

Shu, Gu and Enck studied the security of images distributed through Docker Hub. They analysed a large collection of images and found that many, including official images, contained outdated packages with known vulnerabilities, which motivates scanning images as part of the build rather than trusting the image source.[8] Trivy is an open-source scanner that inspects container images (operating-system packages and language-specific libraries) and matches what it finds against vulnerability databases, reporting severity levels such as CRITICAL and HIGH.[12] Its options include a severity filter, an exit-code setting for failing a build, and an

option to ignore vulnerabilities for which no fixed version exists. Each of these options changes what “pass” means and therefore has to be stated when results are reported.

2.5 Pipelines as Enforcement Points

A pipeline is only an enforcement point if later stages depend on earlier ones. GitHub Actions workflows consist of jobs made of steps; a step that returns a non-zero exit status fails its job, and a job can declare that it needs another job so that it starts only when the first succeeds.[14] Credentials for the cloud account are supplied as encrypted repository secrets instead of being written into the code. These documented behaviours are the mechanism that allows the audit job in this project to block the deployment job.

2.6 Summary of Related Studies

Table 2.1: Review of key literature relevant to the project

Author(s), year	Objective	Sample / method	Main findings
Humble and Farley, 2010[2]	Describe automated build, test and release practices	Practitioner book on delivery pipelines	Automated, repeatable pipelines make releases reliable; supports putting checks inside the pipeline
Bass, Weber and Zhu, 2015[1]	Explain DevOps from an architectural perspective	Practitioner and academic text	Automation and architecture jointly determine how quickly changes can be delivered safely
Myrbakken and Colomo-Palacios, 2017[3]	Clarify what DevSecOps means	Multivocal literature review	Definitions vary, but sources agree on integrating and automating security within DevOps
Rajapakse et al., 2022[4]	Identify challenges and solutions when adopting DevSecOps	Systematic review of published studies	Tool integration, handling of tool output and organisational factors are key adoption challenges
Rahman et al., 2019[6]	Map the state of IaC research	Systematic mapping study	IaC research is growing; quality and security of IaC needs more practical tooling
Rahman, Parnin and Williams, 2019[7]	Find security smells in IaC scripts	Static analysis of open-source IaC scripts	Recurring weaknesses such as hard-coded secrets are common; static analysis can detect them
Shu, Gu and Enck, 2017[8]	Assess vulnerabilities in Docker Hub images	Large-scale scan of public images	Many images, including official ones, contain outdated and vulnerable packages

2.7 Comparative Review of the Tools Used

Table 2.2: Comparison of the security tools used in the pipeline

Tool	Inspects	How it is used in this project	Limitation to keep in mind
Checkov	Terraform and other IaC files, evaluated against policy rules	Step 4 of the audit job; fails the job on any non-suppressed failed check	Only evaluates the files and rules it is given; suppressed checks are not evaluated
TFSec	Terraform (HCL) code	Step 5; second static scan, fails the job on findings	Rule set overlaps with Checkov but is not identical
Trivy	Container image packages and libraries	Step 7; scans the image built in Step 6, fails on CRITICAL/HIGH	Only known, published vulnerabilities; unfixed ones are ignored by the configured option
Terraform validate	Syntax and internal consistency of the configuration	Step 3; fails the job on invalid configuration	Does not check security or whether the plan is wise

2.8 Research Gaps

The reviewed sources establish that DevSecOps is desirable, that IaC and container images contain recurring security weaknesses, and that open-source scanners exist for both. Based on this review, three gaps motivate the proposed work. These are stated as gaps in the sources reviewed for this report, not as claims about all published research.

1. **Integration with a deployment decision.** Studies and tool documentation describe scanners individually, but practical, readable examples that wire IaC scanning, container scanning and a conditional cloud deployment into one small workflow, and then show it working end to end, are less often documented in detail.
2. **Transparency of what "passed" means.** Scan results depend on tool versions, enabled rules and suppression lists. Pipeline demonstrations frequently report a green status without stating which checks were skipped, which risks overstating compliance.
3. **Evidence of both failing and passing behaviour.** A gate is only credible if it has been seen to fail on insecure input as well as pass on secure input. Many demonstrations show only the passing case.

The proposed work addresses these gaps by implementing the complete workflow, by listing every suppressed check and discussing its effect, and by recording a failing run on an intentionally insecure configuration alongside the passing run on the hardened configuration.

Chapter-3: Implementation of Project

This chapter explains how the pipeline was built: the approach followed, the tools and versions used, the structure of the workflow, the configuration of each security gate, and the design of the tests used to evaluate it. All configuration details quoted here are taken from the workflow file devsecops.yml used in the project (reproduced as Scheme 3.2) and from the recorded run logs.

3.1 Objectives of the Implementation

The implementation work follows the objectives O1 to O5 in Table 1.1. In technical terms the pipeline must: (a) start automatically on a push to the main branch; (b) validate the Terraform configuration; (c) evaluate it against security policies with two independent static scanners; (d) build and scan a container image; (e) deploy only when every earlier check has passed; and (f) expose enough information in its logs to show what happened at each stage. No hypothesis testing is involved; the evaluation is by observation of functional behaviour, as described in Section 3.11.

3.2 Methodology

The project was carried out in five phases, summarised in Table 3.1. The phases were iterative: the negative test of Phase 3 was run first against an insecure configuration, and the configuration and workflow were then corrected in Phase 4 until the pipeline passed.

Table 3.1: Phases of the implementation

Phase	Activity	Description
1	Design	Decide the stages of the pipeline, the gate behaviour of each tool and the dependency between the audit and deployment jobs.
2	Build	Write the Terraform configuration, a Dockerfile for the audit image and the GitHub Actions workflow; store AWS credentials as repository secrets.
3	Negative test	Push an intentionally insecure Terraform configuration (public-read S3 bucket and SSH open to the internet) and observe whether the policy gate fails the workflow.
4	Harden and re-run	Correct the configuration, review the Checkov output, record the checks that are deliberately suppressed, and push again until the audit job passes.

Phase	Activity	Description
5	Deploy and verify	Allow the deployment job to run, apply the Terraform configuration in ap-south-1 and confirm that the web server responds at the printed public IP address.

3.3 Technology Stack and Versions

Table 3.2 lists the components and the versions or references that appear in the workflow file and in the run logs. Where the workflow uses a moving reference such as @master instead of a fixed version, this is stated, because it affects repeatability (see Section 3.12).

Table 3.2: Technology stack and versions used

Component	Version / reference	Role
GitHub Actions runner	ubuntu-latest	Executes both jobs
actions/checkout	v3	Retrieves the repository contents
hashicorp/setup-terraform	v2 (Terraform 1.5.0)	Installs the Terraform CLI
Checkov	bridgecrewio/checkov-action@master; Checkov 3.3.29 in the recorded log	Policy-as-code analysis of Terraform
TFSec	aquasecurity/tfsec-action v1.0.0	Static analysis of Terraform
Trivy	aquasecurity/trivy-action@master	Container image vulnerability scan
Docker	Pre-installed on the runner	Builds the image that Trivy scans
aws-actions/configure-aws-credentials	v2	Supplies AWS credentials from repository secrets
Amazon Web Services	Region ap-south-1 (Mumbai)	Target cloud for the deployed web server

3.4 Repository Organisation

The repository (devsecops-pipeline-capstone) keeps the three kinds of artefact that the pipeline works on in separate locations, so that each tool is pointed at exactly the files it should examine. This is summarised in Table 3.3.

Table 3.3: Repository organisation

Location	Content and purpose
Workflow file (devsecops.yml)	GitHub Actions workflow with the audit job and the deployment job; triggered on push to main.
terraform/	Terraform configuration (main.tf) describing the AWS resources; the directory that Terraform, Checkov and TFSec operate on.
docker/	Dockerfile for the container image that is built and scanned in the audit job.

3.5 Pipeline Architecture

Figure 3.1 shows the architecture. A push to the main branch starts the workflow. The first job, named “Shift-Left Static & Vulnerability Auditing” (identifier security-pipeline), performs all validation and scanning. The second job, “Automated Cloud Deployment” (identifier deploy-infrastructure), declares needs: security-pipeline, so GitHub Actions starts it only if the first job has finished successfully. The deployment job additionally carries the condition that the reference is refs/heads/main.

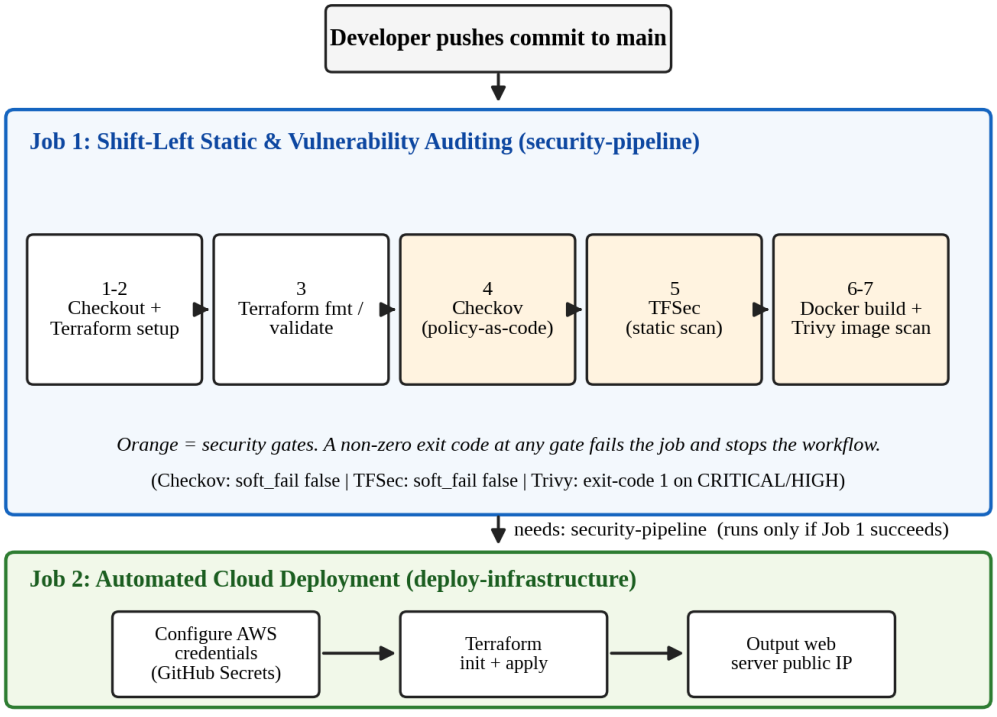


Figure 3.1: Architecture of the automated DevSecOps pipeline

The design follows the principle of failing early and cheaply. The inexpensive static checks run first, the image build and scan follow, and only then is the cloud touched. A failure at any gate

ends the job, and the later steps of that job are skipped by the runner; the deployment job is never scheduled. The audit job also has a 15-minute timeout so that a hung step cannot hold the pipeline indefinitely.

3.6 Job 1: Static and Vulnerability Auditing

Table 3.4 describes each step of the audit job exactly as configured. Steps 4, 5 and 7 are the security gates.

Table 3.4: Steps of the audit job and their configuration

Step	Name	Tool and key parameters	Gate behaviour
1	Checkout Source Code	actions/checkout@v3	Fails only if the repository cannot be retrieved
2	Initialize Terraform CLI	hashicorp/setup-terraform@v2, Terraform 1.5.0	Fails if the CLI cannot be installed
3	Terraform Lint and Validation	terraform fmt -check || true; terraform init -backend=false; terraform validate	Formatting differences do not fail the job (|| true); an invalid configuration does
4	Checkov Policy-as-Code Analysis	checkov-action; file terraform/main.tf; framework terraform; output cli; soft_fail false; skip_check with 13 IDs	Any failed, non-skipped check gives a non-zero exit code and fails the job
5	TFSec Static Infrastructure Scan	tfsec-action v1.0.0; working_directory terraform; soft_fail false	Findings fail the job
6	Build Container Image for Audit	docker build -t target-service:audit-<commit SHA> -f docker/Dockerfile docker/	Fails if the image cannot be built
7	Trivy Container Vulnerability Scan	trivy-action; format table; exit-code 1; ignore-unfixed true; vuln-type os,library; severity CRITICAL,HIGH	Any CRITICAL or HIGH finding that has a fix fails the job

3.6.1 Terraform validation

Before security analysis, the workflow runs terraform init with the remote backend disabled (-backend=false) and then terraform validate. Disabling the backend means that the audit job does not need AWS credentials at all, which is a deliberate separation: the job that analyses code holds no cloud secrets. The terraform fmt -check command is followed by || true, so

formatting differences are reported but do not block the pipeline; syntax and consistency errors found by terraform validate do.

3.6.2 Checkov policy-as-code analysis

Checkov is run through its GitHub Action against terraform/main.tf with the Terraform framework selected and command-line output. The parameter soft_fail is false, so any failed check makes the step, and therefore the job, fail. Each result is reported with its rule identifier (for example CKV_AWS_20), the resource name, the file and the line range, which is what makes the output actionable for the developer. The step also uses a skip_check list, discussed in Section 3.7.

3.6.3 TFSec static scan

TFSec is run on the terraform working directory with soft_fail set to false. It provides a second, independently maintained rule set over the same configuration. In the recorded successful run the step completed without findings.

3.6.4 Container build and Trivy scan

The image is built from docker/Dockerfile and tagged target-service:audit followed by the commit SHA, so that every scan result can be traced to the exact commit that produced the image. Trivy then scans that image. The configuration is intentionally strict on severity (CRITICAL and HIGH) and uses exit-code 1, so a finding at those levels fails the job. Two parameters narrow what is reported, and both should be understood when reading results: ignore-unfixed true excludes vulnerabilities for which no patched version exists, and vuln-type os,library limits the scan to operating-system packages and application libraries.

3.7 Checkov Check Suppression

The Checkov step in the final workflow carries a skip_check parameter listing thirteen rule identifiers. Table 3.5 states what each rule examines. Suppressing a rule means it is not evaluated at all, so the passing result of the Checkov step must be read as “all checks other than these thirteen passed”.

Table 3.5: Checkov checks suppressed in the final workflow

Rule ID	What the rule examines	Area
CKV_AWS_18	S3 bucket has access logging enabled	S3 logging
CKV_AWS_144	S3 bucket has cross-region replication enabled	S3 resilience

Rule ID	What the rule examines	Area
CKV2_AWS_61	S3 bucket has a lifecycle configuration	S3 lifecycle
CKV2_AWS_62	S3 bucket has event notifications enabled	S3 events
CKV2_AWS_5	Security group is attached to another resource	Network hygiene
CKV_AWS_260	No security group allows ingress from 0.0.0.0/0 to port 80	Network exposure
CKV_AWS_382	No security group allows egress to 0.0.0.0/0 on all ports	Network exposure
CKV_AWS_135	EC2 instance is EBS-optimised	EC2 configuration
CKV_AWS_79	Instance Metadata Service version 1 is not enabled	EC2 hardening
CKV_AWS_8	Data on the instance's EBS storage is encrypted	EC2 encryption
CKV_AWS_126	EC2 instance has detailed monitoring enabled	EC2 monitoring
CKV_AWS_88	EC2 instance does not have a public IP address	EC2 exposure
CKV2_AWS_41	An IAM role is attached to the EC2 instance	EC2 identity

Two points follow from the table. First, two suppressions (CKV_AWS_260 for HTTP on port 80 and CKV_AWS_88 for a public IP address) are consistent with the purpose of the demonstration, which is a public web server whose address is printed by the pipeline and opened in a browser. Second, several suppressed rules (for example CKV_AWS_8 on storage encryption and CKV_AWS_79 on the metadata service) concern hardening that would be expected in a production deployment. They are suppressed in the final workflow, which is consistent with keeping the demonstration to a single low-cost instance, and they are listed again as items to enable in the future scope. The report does not claim compliance with any benchmark for the suppressed areas.

3.8 Job 2: Automated Cloud Deployment

The deployment job runs on ubuntu-latest and executes the steps in Table 3.6. Its credentials are read from the repository secrets AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY and are never written in the workflow file. The region is fixed to ap-south-1.

Table 3.6: Steps of the deployment job

Step	Name	Action
1	Checkout Source Code	Retrieves the repository (actions/checkout@v3).
2	Configure AWS Credentials	aws-actions/configure-aws-credentials@v2 with the two secrets and region ap-south-1.
3	Setup Terraform CLI	Installs Terraform 1.5.0.
4	Terraform Apply	cd terraform; terraform init; terraform apply -auto-approve. State reconciliation against the stored state decides what is created or changed.
5	Display Live Web Server Public IP	Prints the URL built from terraform output -raw web_server_public_ip.

The use of -auto-approve is acceptable here because the job can only start after the security gates have passed and the target is a single demonstration resource set. It is not a pattern recommended for production, where a reviewed plan and a manual approval would normally be required; this is listed under future scope.

3.9 State Management and Repeatable Provisioning

Terraform records which real resources correspond to the declared ones in a state file. If the state were kept only on the temporary GitHub runner, every run would start with an empty state and would try to create the resources again, producing duplicates or name conflicts. The deployment job therefore initialises Terraform with its backend enabled, so that successive runs reconcile against the same stored state; the audit job, by contrast, uses -backend=false because it performs no deployment. The project's configuration stores the remote state in an Amazon S3 bucket in ap-south-1. Because state files can contain sensitive values, the bucket should be private, encrypted and versioned, and the report does not publish its contents.

3.10 Gate Logic

Scheme 3.1 expresses the control flow of the workflow in pseudocode. It is a description of the behaviour implemented by the workflow file, not additional code.

Scheme 3.1: Gate logic of the DevSecOps workflow

```
ON push to main:
  JOB security-pipeline:
    checkout; install Terraform 1.5.0
    terraform init -backend=false; terraform validate
    IF checkov(terraform/main.tf, 13 checks skipped) fails THEN FAIL
    IF tfsec(terraform/) finds issues THEN FAIL
```

```
    build image target-service:audit-<commit SHA>
    IF trivy(image, CRITICAL/HIGH, ignore-unfixed) finds any THEN FAIL
    (a FAIL ends the job; remaining steps are skipped)
  JOB deploy-infrastructure:
    RUN only IF security-pipeline succeeded AND ref == main
    configure AWS credentials (secrets), region ap-south-1
    terraform init; terraform apply -auto-approve
    print http://<web_server_public_ip>
```

3.11 Test Design

Two kinds of test were designed. The negative test checks that the gate fails when it should. The positive test checks that the pipeline passes, and deploys, when the configuration is acceptable. The negative test uses an intentionally insecure Terraform configuration containing two resources, summarised in Table 3.7; the line numbers refer to main.tf in the run being tested.

Table 3.7: Intentionally insecure test resources used for the negative test

Resource	Insecure property	Lines in main.tf
aws_s3_bucket.financial_data_lake (bucket “enterprise-confidential-financial-records-2026”)	acl = "public-read"; no versioning, encryption with KMS, access logging, public access block or lifecycle rules defined	18–25
aws_security_group.management_ssh (name “allow_management_ssh”)	Ingress on TCP port 22 from cidr_blocks 0.0.0.0/0; rule has no description	28–38

The expected result of the negative test is a failed Checkov step with exit code 1, skipped later steps and no deployment. The expected result of the positive test is a successful audit job followed by a successful deployment job and a web page reachable at the printed address. In the failing runs the deployment job did not execute, so the insecure resources were analysed as code only and were never created in AWS.

3.12 Security of the Pipeline Itself

A pipeline that enforces security must itself be handled securely. The following design points were applied or identified during the work:

- **Secrets.** AWS keys are held as encrypted repository secrets and are referenced, never written, in the workflow. The audit job receives no cloud credentials.
- **Separation of duties.** Analysis and deployment are separate jobs, and the second cannot start unless the first succeeds.

- **Traceability.** Image tags include the commit SHA, and every run is logged by GitHub with the commit and actor.
- **Moving references.** Two actions (checkov-action and trivy-action) are referenced by @master, and others by major version tags. A new release of an action can therefore change pipeline behaviour without any change in the repository. Pinning to a specific version or commit SHA is recommended for future work.
- **Long-lived keys.** Static access keys stored as secrets are less safe than short-lived credentials obtained through GitHub's OpenID Connect federation with an AWS role; this is listed as future scope.
- **Platform notices.** The run annotations show a Node.js 20 deprecation notice for actions/checkout@v3, hashicorp/setup-terraform@v2 and aws-actions/configure-aws-credentials@v2, and a notice that the ubuntu-latest label moves to a newer Ubuntu release from 19 October 2026. These do not affect the results reported here but show why action versions need periodic maintenance.

3.13 Workflow Definition

Scheme 3.2 reproduces the workflow definition used in the project. It contains no secret values; the credentials are references to repository secrets.

Scheme 3.2: Workflow definition (devsecops.yml)

```
name: "Automated DevSecOps Pipeline"

on:
  push:
    branches: [ "main" ]

jobs:
  security-pipeline:
    name: "Shift-Left Static & Vulnerability Auditing"
    runs-on: ubuntu-latest
    timeout-minutes: 15

    steps:
      - name: "Step 1: Checkout Source Code"
        uses: actions/checkout@v3

      - name: "Step 2: Initialize Terraform CLI"
        uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: 1.5.0

      - name: "Step 3: Terraform Lint and Validation"
        run: |
          cd terraform
          terraform fmt -check || true
          terraform init -backend=false
          terraform validate

      - name: "Step 4: Checkov Policy-as-Code Analysis"
        uses: bridgecrewio/checkov-action@master
        with:
          file: terraform/main.tf
          framework: terraform
```

```
          output_format: cli
          skip_check:
"CKV_AWS_18,CKV_AWS_144,CKV2_AWS_61,CKV2_AWS_62,CKV2_AWS_5,CKV_AWS_260,CKV_AWS_135,CKV_AWS_79,CKV
_AWS_8,CKV_AWS_126,CKV_AWS_88,CKV_AWS_382,CKV2_AWS_41"
          soft_fail: false

      - name: "Step 5: TFSec Static Infrastructure Scan"
        uses: aquasecurity/tfsec-action@v1.0.0
        with:
          working_directory: terraform
          soft_fail: false

      - name: "Step 6: Build Container Image for Audit"
        run: |
          docker build -t target-service:audit-${{ github.sha }} -f docker/Dockerfile docker/

      - name: "Step 7: Trivy Container Vulnerability Scan"
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: "target-service:audit-${{ github.sha }}"
          format: "table"
          exit-code: "1"
          ignore-unfixed: true
          vuln-type: "os,library"
          severity: "CRITICAL,HIGH"

  deploy-infrastructure:
    name: "Automated Cloud Deployment"
    needs: security-pipeline
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'

    steps:
      - name: "Checkout Source Code"
        uses: actions/checkout@v3

      - name: "Configure AWS Credentials"
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws-secret-access-key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws-region: "ap-south-1"

      - name: "Setup Terraform CLI"
        uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: 1.5.0

      - name: "Terraform Apply (Single Resource State Reconciliation)"
        run: |
          cd terraform
          terraform init
          terraform apply -auto-approve

      - name: "Display Live Web Server Public IP"
        run: |
          cd terraform
          echo "============================================================"
          echo "DEPLOYMENT COMPLETE!"
          echo "Web Server URL: http://$(terraform output -raw web_server_public_ip)"
          echo "============================================================"
```

Chapter-4: Results and Discussions

This chapter presents the results obtained from the recorded pipeline runs. The evidence consists of the GitHub Actions run summaries and job views, the Checkov console log of the negative test, the workflow definition, and a screenshot of the deployed web page. Every figure and number below is taken from that evidence; where something was not captured, the chapter says so.

4.1 Evidence Sources

Two groups of runs were recorded. The first group consists of failed audit runs on the insecure configuration (two job views with 11 and 10 error annotations respectively, and one step-level view). The second group is a successful run on the hardened configuration, triggered by a push to the main branch by the repository owner, in which both jobs completed. The Checkov console log (Checkov version 3.3.29) of the failed run is available as text and gives the complete list of passed and failed checks.

4.2 Functional Test Results

Table 4.1 summarises the functional tests against the expected results defined in Section 3.11.

Table 4.1: Functional test results

ID	Test	Expected result	Observed result
T1	Workflow triggers on push to main	Workflow starts automatically	Run summary shows “Triggered via push” for a commit on main; status Success
T2	Terraform validation	Step completes for a valid configuration	Step 3 completed in the audit job in the failed and in the successful runs
T3	Negative test: policy gate on insecure Terraform	Checkov fails, job stops, no deployment	Checkov step failed with exit code 1; 11 checks failed and 8 passed; steps 5–7 skipped; deployment job did not run
T4	Positive test: audit job on hardened Terraform	All seven audit steps succeed	All steps green in the audit job; job duration 43 s
T5	Deployment after successful audit	Deployment job starts and succeeds	Deployment job succeeded in 45 s; Terraform apply step green
T6	Live endpoint	Web page reachable at printed public IP	Browser shows the “Automated DevSecOps Pipeline Showcase” page served from the public IP

4.3 Negative Test: Policy-as-Code Interception

The insecure configuration of Table 3.7 was pushed to the repository. The Checkov step evaluated 19 checks on the two resources: 8 passed and 11 failed. Because soft_fail was false, the step ended with exit code 1 and the job was marked failed. Table 4.2 lists every check from the log.

Table 4.2: Checkov results for the insecure configuration (8 passed, 11 failed)

Rule ID	Check description	Resource	Result
CKV_AWS_20	S3 bucket ACL allows public READ access	S3 bucket	FAILED
CKV_AWS_24	No security group allows ingress from 0.0.0.0/0 to port 22	Security group	FAILED
CKV_AWS_23	Every security group and rule has a description	Security group	FAILED
CKV2_AWS_6	S3 bucket has a Public Access Block	S3 bucket	FAILED
CKV_AWS_145	S3 bucket is encrypted with KMS by default	S3 bucket	FAILED
CKV_AWS_21	S3 bucket has versioning enabled	S3 bucket	FAILED
CKV_AWS_18	S3 bucket has access logging enabled	S3 bucket	FAILED
CKV_AWS_144	S3 bucket has cross-region replication enabled	S3 bucket	FAILED
CKV2_AWS_61	S3 bucket has a lifecycle configuration	S3 bucket	FAILED
CKV2_AWS_62	S3 bucket has event notifications enabled	S3 bucket	FAILED
CKV2_AWS_5	Security group is attached to another resource	Security group	FAILED
CKV_AWS_41	No hard-coded AWS access key and secret key in provider	AWS provider	PASSED
CKV_AWS_93	S3 bucket policy does not lock out all but the root user	S3 bucket	PASSED
CKV_AWS_57	S3 bucket ACL does not allow public WRITE access	S3 bucket	PASSED
CKV_AWS_19	Data in the S3 bucket is encrypted at rest	S3 bucket	PASSED
CKV_AWS_25	No ingress from 0.0.0.0/0 to port 3389	Security group	PASSED
CKV_AWS_260	No ingress from 0.0.0.0/0 to port 80	Security group	PASSED
CKV_AWS_277	No ingress from 0.0.0.0/0 to all ports (-1)	Security group	PASSED
CKV_AWS_382	No egress to 0.0.0.0/0 on all ports (-1)	Security group	PASSED

The two findings that motivated the test, CKV_AWS_20 (public-read ACL) and CKV_AWS_24 (SSH open to 0.0.0.0/0), were both detected. The log also shows the offending source lines (lines 18–25 and 28–38 of main.tf) with each finding, which lets a developer go straight to the resource that needs correcting. Figure 4.1 shows the annotations as displayed by GitHub for the failed run, and Figure 4.2 shows the step-level view in which only Step 4 failed and Steps 5 to 7 were not executed.

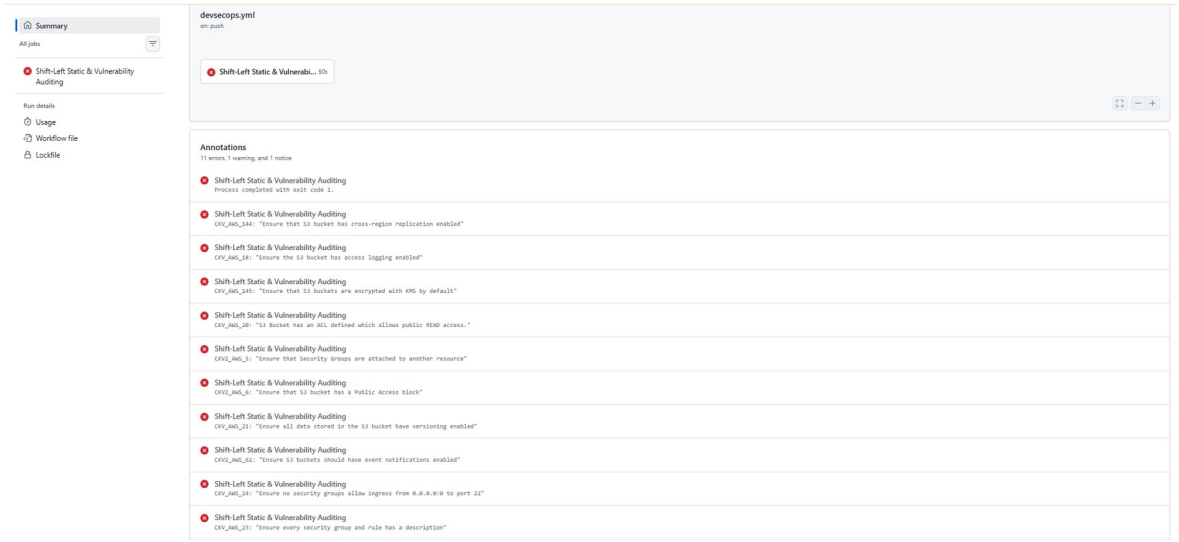


Figure 4.1: Failed audit run: Checkov annotations in GitHub Actions (11 errors)

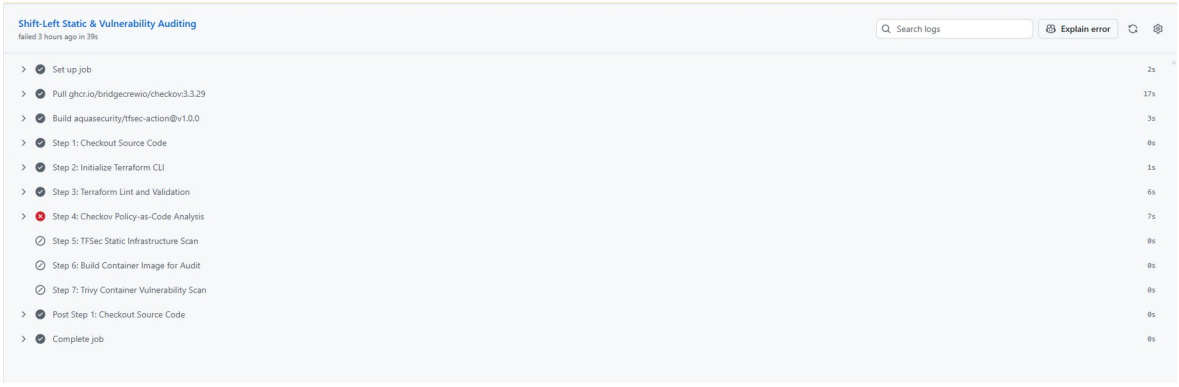


Figure 4.2: Failed audit job: Step 4 (Checkov) failed; Steps 5 to 7 skipped

Another failed run that was recorded shows the same pattern with 10 error annotations (Figure 4.3). The annotation counts of the two views differ (11 and 10); the underlying Checkov outcome is the one given in Table 4.2.

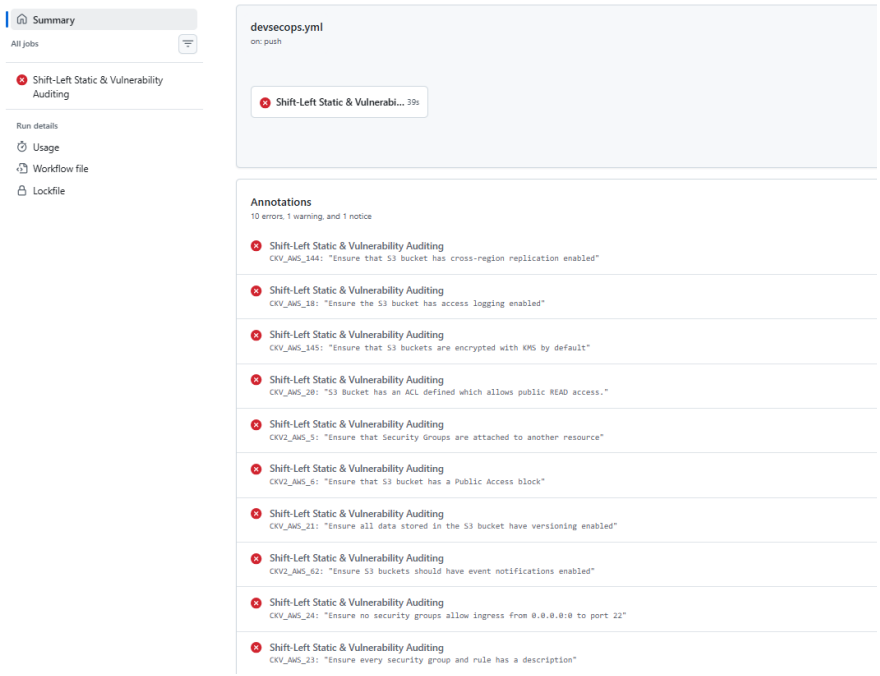


Figure 4.3: Another failed audit run: Checkov annotations (10 errors)

The most important result of this test is not the number of findings but the behaviour of the workflow around them. The failed jobs lasted 39 and 50 seconds, the TFSec, image build and Trivy steps were skipped, and the deployment job never started. No AWS resource was created from the insecure code. This answers the second research question affirmatively for the tested case.

4.4 Positive Test: Hardened Configuration

After the configuration was corrected and the workflow was updated (including the suppression list of Table 3.5), a push to the main branch produced a successful run. Figure 4.4 shows the run summary: both jobs succeeded and the total duration was 1 minute 33 seconds.

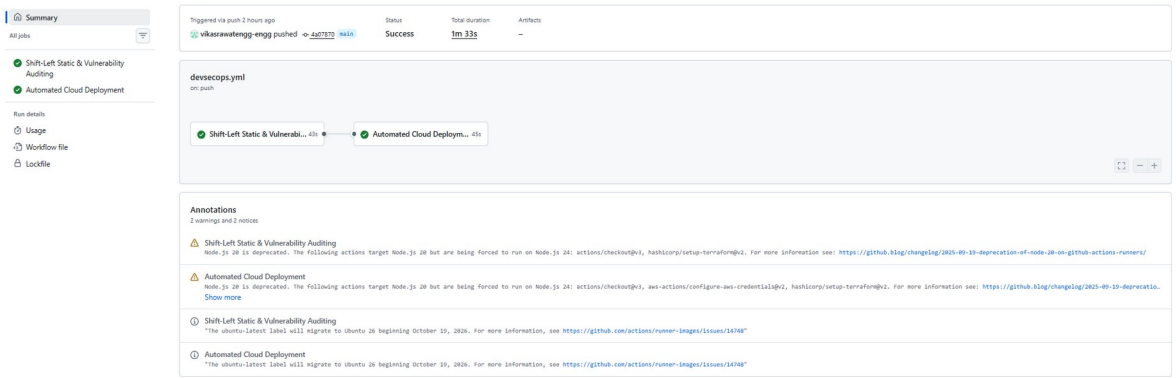


Figure 4.4: Successful workflow run summary (audit job followed by deployment job)

Figure 4.5 shows the job views. In the audit job all seven numbered steps, together with the preparatory steps that pull the Checkov image and build the TFSec action, completed

successfully; in the deployment job the checkout, credential, Terraform and IP-display steps also completed. In the audit job, Step 7 (Trivy) completed with exit-code 1 configured, which means that no CRITICAL or HIGH vulnerability with an available fix was found in the image under the options listed in Section 3.6.4. The Trivy output table itself was not captured, so no vulnerability counts are reported.

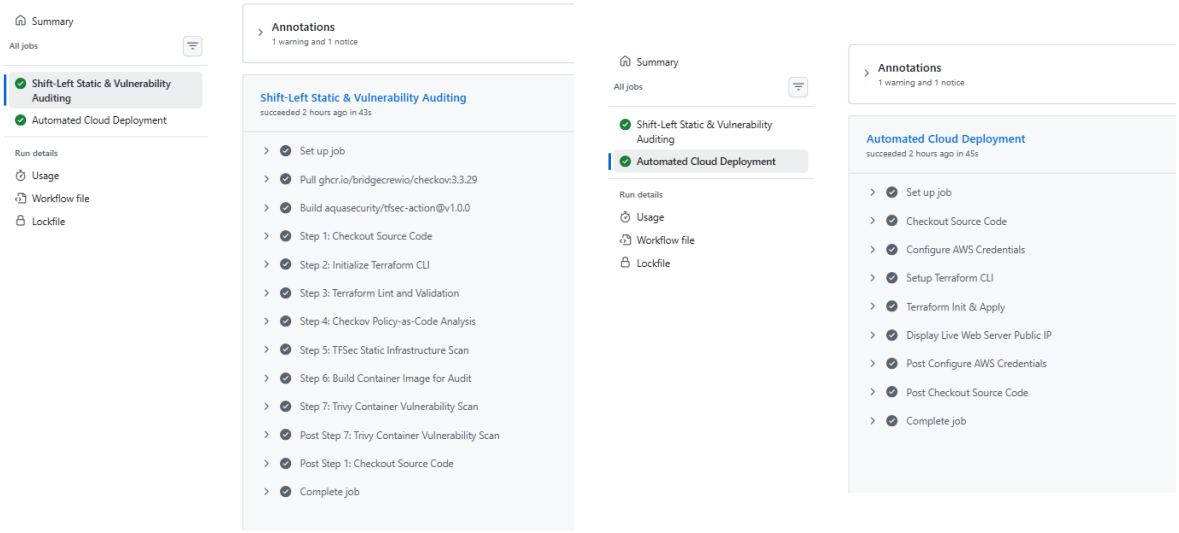


Figure 4.5: Job views of the successful run: (a) audit job, (b) deployment job

The run annotations (visible in Figure 4.4) contain two warnings and two notices: a Node.js 20 deprecation warning for three actions and a notice about the ubuntu-latest runner label moving to a newer Ubuntu release. They are maintenance items rather than security results.

4.5 Automated Deployment and Live Endpoint

The deployment job applied the Terraform configuration in ap-south-1 and printed the public address of the web server. Opening that address in a browser shows the project's showcase page (Figure 4.6), which displays the student identification, the project title and a "Pipeline Gate Verification Matrix". The page confirms that the deployed instance is serving content over HTTP; it is a project-generated page and is not itself an independent certification of the scan results.

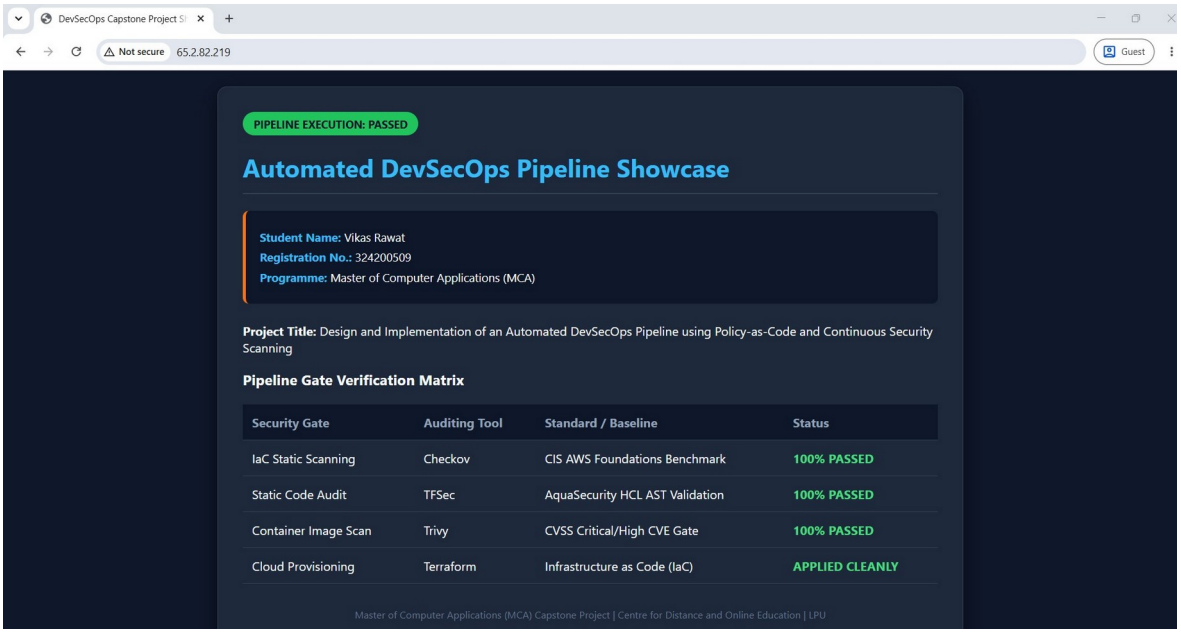


Figure 4.6: Live web server deployed by the pipeline (showcase page)

The matrix on the page labels the Checkov gate as “100% PASSED” against the CIS AWS Foundations Benchmark. This wording should be read with care. The Checkov step passed all checks that were evaluated, but thirteen checks were suppressed (Table 3.5) and the tool does not evaluate every CIS recommendation. The statement therefore means “the configured Checkov checks passed”, not “the deployment is fully CIS compliant”. The same caution applies to the other rows of the matrix.

4.6 Pipeline Timing

Table 4.3 and Figure 4.7 give the durations visible in the recorded runs. They are single observations on GitHub-hosted runners and are not averages; durations vary with runner load and with the time needed to download tool images.

Table 4.3: Observed job durations

Run / job	Duration	Outcome
Failed audit run (view with 11 error annotations)	50 s	Failed at Checkov (Step 4)
Failed audit run (view with 10 error annotations)	39 s	Failed at Checkov (Step 4)
Successful audit job	43 s	All seven steps passed
Successful deployment job	45 s	Terraform applied; IP displayed
Successful run, total	1 min 33 s	Success

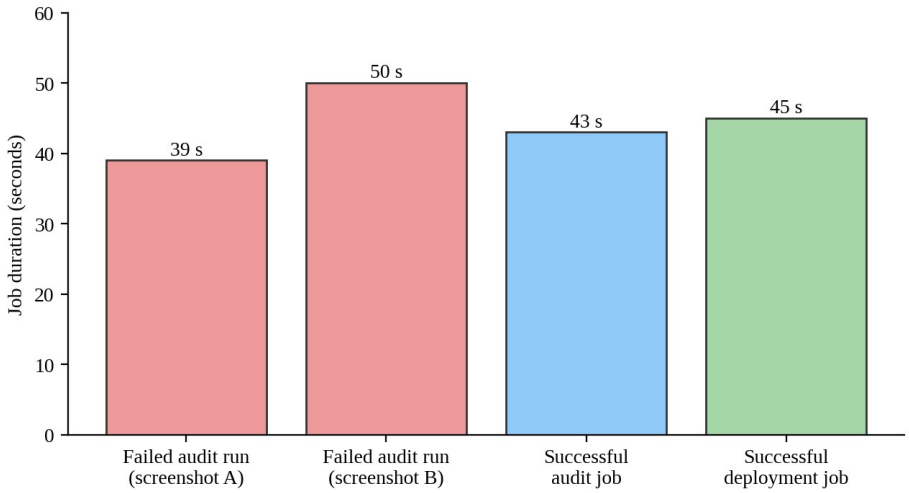


Figure 4.7: Observed job durations from the recorded runs

Two observations follow. First, the complete audit job, including pulling tool images, running three scanners and building and scanning a container image, finished in 43 seconds in the successful run, which is a small addition to a push-to-deploy cycle of 1 minute 33 seconds. Second, in the failed run the step view shows that Checkov itself ran for 7 seconds, and that the job stopped there; failing early avoided the time that the skipped steps would have used. Because only job-level durations are visible for the successful run, no claims are made about the duration of individual scanners in that run.

4.7 Mapping of Findings to Remediation

Table 4.4 relates each failed check of the negative test to the usual corrective action and to its status in the final workflow. The status is derived from the workflow file: a check that appears in the skip list is recorded as suppressed; a check that does not appear in it must have passed or no longer applied, because the Checkov step succeeded with soft_fail false.

Table 4.4: Failed checks, corrective action and status in the final workflow

Rule ID	Typical corrective action	Status in the final workflow
CKV_AWS_20	Remove the public-read ACL; keep the bucket private	Not suppressed; no failure in successful run
CKV_AWS_24	Restrict SSH to a trusted CIDR, use a bastion or Session Manager, or remove the rule	Not suppressed; no failure in successful run
CKV_AWS_23	Add a description to the group and each rule	Not suppressed; no failure in successful run
CKV2_AWS_6	Add an S3 public access block with all four settings enabled	Not suppressed; no failure in successful run

Rule ID	Typical corrective action	Status in the final workflow
CKV_AWS_145	Enable default encryption with a KMS key	Not suppressed; no failure in successful run
CKV_AWS_21	Enable bucket versioning	Not suppressed; no failure in successful run
CKV_AWS_18	Enable server access logging to a protected log bucket	Suppressed (Table 3.5)
CKV_AWS_144	Configure cross-region replication where recovery objectives require it	Suppressed (Table 3.5)
CKV2_AWS_61	Add a lifecycle configuration	Suppressed (Table 3.5)
CKV2_AWS_62	Add event notifications to an approved destination	Suppressed (Table 3.5)
CKV2_AWS_5	Attach the security group to a resource or delete it	Suppressed (Table 3.5)

Six of the eleven failures are neither suppressed nor reported in the successful run, which means the configuration was corrected or the resource no longer applies; these include the two high-impact exposures (public bucket and open SSH). The five suppressed items are operational or resilience controls (logging, replication, lifecycle rules, notifications and attachment of the security group) that generally require additional AWS resources or design decisions beyond a single-instance demonstration. They remain open items for a production version.

4.8 Achievement of Objectives

Table 4.5: Achievement of project objectives

Objective	Evidence	Status
O1	Workflow runs on push to main (run summary)	Achieved
O2	Checkov and TFSec configured as blocking gates; Checkov failure observed on insecure code; both passed on hardened code	Achieved; TFSec failure on insecure code was not separately captured
O3	Image built and Trivy step configured with exit-code 1 and CRITICAL/HIGH; step passed in the successful run	Partly achieved; no failing Trivy run and no Trivy output captured
O4	Deployment job depends on audit job; deployment succeeded; web page reachable; deployment did not run in failed runs	Achieved

Objective	Evidence	Status
O5	Negative and positive runs recorded; timing and remediation mapping prepared	Achieved for Checkov; limited for Trivy and TFSec

4.9 Discussion

What the results show. The pipeline behaves as designed at its main control point. An insecure Terraform definition is detected by Checkov, reported with rule identifiers and source lines, and the workflow stops before the deployment job is scheduled. A corrected definition passes all stages and is deployed within about a minute and a half of the push. This is the central claim of the project and it is supported directly by the recorded runs.

What a green run means. A successful run means that every enabled gate returned a zero exit status. It does not mean that the deployment is free of vulnerabilities or compliant with a benchmark. The suppression list, the ignore-unfixed option of Trivy and the limited rule coverage of any scanner all narrow what "passed" covers. For this reason the report states the configuration of each gate and the suppressed checks explicitly.

Early detection is cheap. The failing runs lasted well under a minute and consumed no cloud resources, whereas the equivalent problems found after deployment would require changing live infrastructure. The result agrees with the shift-left argument of the literature,[3,4] though the project does not measure the cost of defects and therefore makes no quantitative claim about savings.

Suppression is a design decision. The thirteen suppressed checks are the main compromise in the final configuration. They keep a deliberately small deployment within scope, but in a production setting each would need either remediation or a documented, time-limited exception. Recording the list openly is itself part of the result, because it addresses the second research gap in Chapter-2.

4.10 Limitations and Threats to Validity

- **Single environment and few runs.** The results come from a small number of runs in one repository; they illustrate behaviour but are not a statistical evaluation.
- **Negative test scope.** The failing behaviour was recorded for the Checkov gate only. A deliberately vulnerable container image for Trivy and an insecure input for TFSec were

not recorded, so the blocking behaviour of those two gates is configured but not demonstrated by a failing run.

- **No Trivy output.** The vulnerability table was not captured, so counts of vulnerabilities, the base image and the database date are not reported.
- **Repeat-run behaviour.** Reconciliation against stored state is part of the design, but a second apply showing “no changes” was not recorded, so repeatable provisioning is not demonstrated by evidence.
- **Version drift.** Checkov and Trivy actions are referenced by @master; results can change when those projects release new rules or databases.
- **Scope of the audit.** The work is a pipeline demonstration. It is not a penetration test, a runtime assessment or a formal CIS audit.

Chapter-5: Conclusion and Future Scope

5.1 Conclusion

This project designed, implemented and evaluated an automated DevSecOps pipeline in which security checks run on every push and control whether an AWS deployment happens. The pipeline consists of two GitHub Actions jobs. The first validates the Terraform configuration, evaluates it with Checkov and TFSec, builds a container image and scans it with Trivy; the second configures AWS credentials from repository secrets and applies the Terraform configuration only if the first job succeeded.

The evidence recorded in the repository supports the main conclusions. An intentionally insecure configuration containing a public-read S3 bucket and SSH open to the internet was stopped by the Checkov gate, which reported 11 failed and 8 passed checks and ended the job with exit code 1; the later steps were skipped and the deployment job did not run. A corrected configuration passed all audit steps in 43 seconds and was deployed by a deployment job of 45 seconds, for a total run time of 1 minute 33 seconds, after which the deployed web server was reachable in a browser.

The project also draws a practical lesson about interpreting results. A green pipeline means that the configured gates returned success, not that the system is secure. The final workflow suppresses thirteen Checkov checks and configures Trivy to ignore vulnerabilities without a fix, so the claim that can be defended is limited to the checks actually evaluated. Reporting those settings openly, rather than a bare “100% passed”, is what makes the evidence trustworthy.

Against the objectives, O1 and O4 are fully achieved, O2 and O5 are achieved for the Checkov gate, and O3 is partly achieved because the Trivy gate was configured and observed to pass but was not shown failing on a vulnerable image.

5.2 Contributions

- A compact, readable reference workflow that connects IaC validation, two static scanners, container scanning and conditional deployment.
- A documented negative test with complete Checkov results and source-line evidence for an insecure configuration.

- An explicit register of suppressed checks and an interpretation guide for what a passing run does and does not prove.

5.3 Limitations

The limitations are those listed in Section 4.10: a small number of runs in one environment, no failing run for the Trivy and TFSec gates, no captured Trivy output, no recorded repeat apply, use of moving action references and a scope restricted to pipeline behaviour rather than a full security audit.

5.4 Future Scope

1. **Complete the negative tests.** Add a deliberately vulnerable image to demonstrate the Trivy gate failing, an insecure input for TFSec, and a repeated apply that shows no changes.
2. **Reduce suppressions.** Remediate or formally time-limit each of the thirteen suppressed checks, beginning with EBS encryption, the instance metadata service and logging.
3. **Stronger credentials.** Replace static access keys with short-lived credentials through GitHub OpenID Connect and a least-privilege IAM role.
4. **Plan and approval gate.** Add a terraform plan stage with a saved plan, and require manual approval before apply in protected environments.
5. **Pin and update actions.** Pin all third-party actions to fixed versions or commit SHAs and update those flagged as deprecated. Review the long-term status of the TFSec action, since the TFSec project's functionality is being consolidated into Trivy.
6. **Machine-readable reports.** Publish SARIF or JSON scan reports as run artefacts and track findings over time.
7. **Wider coverage.** Add secrets scanning, software bill of materials generation, dependency scanning, dynamic application testing against the deployed endpoint and post-deployment smoke tests.
8. **Multi-cloud and policy library.** Extend the policy set with custom organisation-specific rules and support other cloud providers.

5.5 Applicability

The workflow can be reused as a template by small teams that deploy Terraform to AWS and want an automatic, auditable security gate at low cost. The same structure, with the tool parameters and suppression list adapted, applies to other IaC projects and to other container-based services. For production use it should be extended with the future-scope items above, especially credential hardening, approval gates and a reviewed exception process.

List of Publications / Conference Papers

No research paper, conference paper or book chapter arising directly from this project work has been published as on the date of submission of this report.

Annexures

Annexure - I (Cover Page) is placed as the first page of this report and Annexure - II (Declaration by the Student) as the second page, in the prescribed formats. The signed and scanned copy of Annexure - II is included with the report uploaded on the LMS.

REFERENCES

1. L. Bass, I. Weber, and L. Zhu, DevOps: A Software Architect's Perspective (1st ed.), Addison-Wesley Professional, **2015**.
2. J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation (1st ed.), Addison-Wesley Professional, **2010**.
3. M. Myrbakken and R. Colomo-Palacios, "DevSecOps: A multivocal literature review," *Software Process Improvement and Capability Determination (SPICE 2017), Communications in Computer and Information Science*, vol. **770**, pp. 17–29, 2017.
4. R. N. Rajapakse, M. Zahedi, M. A. Babar, and H. Shen, "Challenges and solutions when adopting DevSecOps: A systematic review," *Information and Software Technology*, vol. **141**, art. no. 106700, 2022.
5. K. Morris, Infrastructure as Code: Dynamic Systems for the Cloud Age (2nd ed.), O'Reilly Media, **2020**.
6. A. Rahman, R. Mahdavi-Hezaveh, and L. Williams, "A systematic mapping study of infrastructure as code research," *Information and Software Technology*, vol. **108**, pp. 65–77, 2019.
7. A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," *Proc. 41st Int. Conf. on Software Engineering (ICSE)*, pp. 164–175, 2019.
8. R. Shu, X. Gu, and W. Enck, "A study of security vulnerabilities on Docker Hub," *Proc. 7th ACM Conf. on Data and Application Security and Privacy (CODASPY)*, pp. 269–280, 2017.
9. https://www.cisecurity.org/benchmark/amazon_web_services; Dated: 19/10/2026
10. https://www.checkov.io; Dated: 19/10/2026
11. https://github.com/aquasecurity/tfsec; Dated: 19/10/2026
12. https://github.com/aquasecurity/trivy; Dated: 19/10/2026
13. https://developer.hashicorp.com/terraform/docs; Dated: 19/10/2026
14. https://docs.github.com/en/actions; Dated: 19/10/2026